

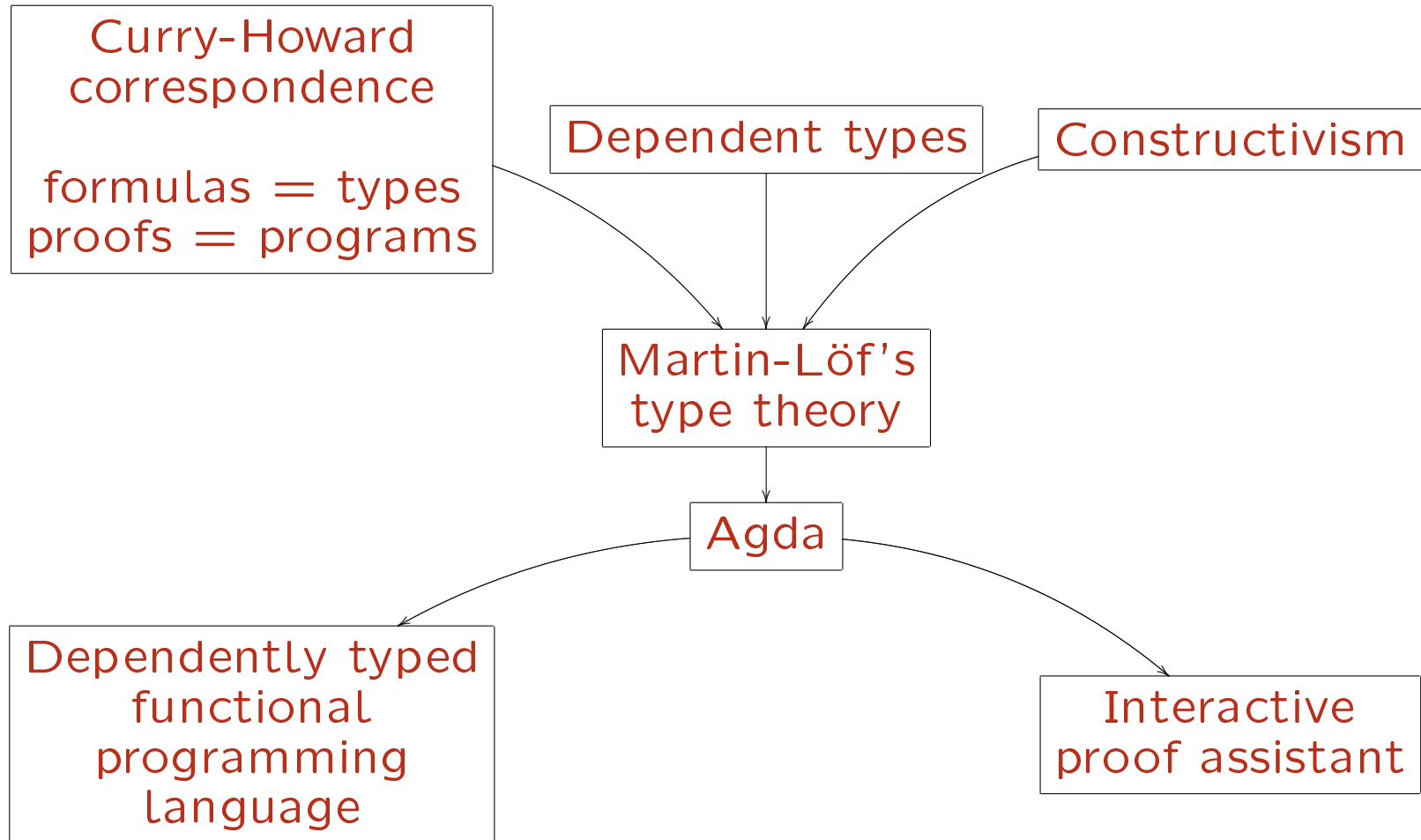
Proofs = Programs

Andrés Sicard-Ramírez

Grupo de Lógica y Computación
Universidad EAFIT

Días de la Ciencia Aplicada
Universidad EAFIT
2009-09-29

Overview



Constructive interpretation of the logical constants

- Proofs are constructions (programs)
- Reject of the principle of the excluded third

The Brouwer-Heyting-Kolmogorov (BHK) interpretation:

A construction of:	Consists of:
$\sigma_1 \wedge \sigma_2$	A construction of σ_1 and a construction of σ_2 .
$\sigma_1 \vee \sigma_2$	An indicator $i \in \{1, 2\}$ and a construction of σ_i .
$\sigma_1 \rightarrow \sigma_2$	A method (function) which takes any construction of σ_1 to a construction of σ_2 .
$\perp$	There is not construction.

Intuitionistic logic: Fragment $NJ(\rightarrow)$

Definition (Judgement).

Γ : finite set of formulas

$\Gamma \vdash \sigma$: Γ proves σ

Definition (Rules).

$$\Gamma, \sigma \vdash \sigma \quad (\text{Ax})$$

$$\frac{\Gamma, \sigma \vdash \tau}{\Gamma \vdash \sigma \rightarrow \tau} \quad (\rightarrow I)$$

$$\frac{\Gamma \vdash \sigma \rightarrow \tau \quad \Gamma \vdash \sigma}{\Gamma \vdash \tau} \quad (\rightarrow E)$$

Example.

$$\frac{\sigma \vdash \sigma}{\vdash \sigma \rightarrow \sigma} \quad (\rightarrow I)$$

Intuitionistic logic: Fragment $NJ(\rightarrow)$ (cont.)

Convention: The implication is right associative

e.g. $\sigma \rightarrow (\tau \rightarrow \sigma) \equiv \sigma \rightarrow \tau \rightarrow \sigma$

Example.

$$\frac{\frac{\sigma, \tau \vdash \sigma}{\sigma \vdash \tau \rightarrow \sigma} (\rightarrow I)}{\vdash \sigma \rightarrow \tau \rightarrow \sigma} (\rightarrow I)$$

Intuitionistic logic: Fragment $NJ(\rightarrow)$ (cont.)

Example. $\Gamma = \{\sigma \rightarrow \tau \rightarrow \rho, \sigma \rightarrow \tau, \sigma\}$

$$\frac{\frac{\frac{\Gamma \vdash \sigma \rightarrow \tau \rightarrow \rho \quad \Gamma \vdash \sigma}{\Gamma \vdash \tau \rightarrow \rho} (\rightarrow E) \quad \frac{\frac{\Gamma \vdash \sigma \rightarrow \tau \quad \Gamma \vdash \sigma}{\Gamma \vdash \tau} (\rightarrow E)}{\Gamma \vdash \rho} (\rightarrow E)}{\frac{\sigma \rightarrow \tau \rightarrow \rho, \sigma \rightarrow \tau \vdash \sigma \rightarrow \rho}{\sigma \rightarrow \tau \rightarrow \rho \vdash (\sigma \rightarrow \tau) \rightarrow \sigma \rightarrow \rho} (\rightarrow I)} (\rightarrow I)$$
$$\frac{\sigma \rightarrow \tau \rightarrow \rho \vdash (\sigma \rightarrow \tau) \rightarrow \sigma \rightarrow \rho}{\vdash (\sigma \rightarrow \tau \rightarrow \rho) \rightarrow (\sigma \rightarrow \tau) \rightarrow \sigma \rightarrow \rho} (\rightarrow I)$$

λ -calculus

Invented by the American mathematician and logician Alonzo Church (around 1930s).

Intended for studying **functions** and **recursion**.



Informally

Element	Example	Denotes
Abstraction	$\lambda x. x^2 + 1$	Function $x \mapsto x^2 + 1$
Application	$(\lambda x. x^2 + 1) 3$	Function $x \mapsto x^2 + 1$ applied to 3
β -reduction	$(\lambda x. x^2 + 1) 3 \rightarrow_{\beta} 3^2 + 1$	The value of function $x \mapsto x^2 + 1$ applied to 3

λ -calculus (cont.)

Definition (λ -terms).

$$\begin{aligned}\Lambda ::= & x && \text{(variables)} \\ & | \Lambda\Lambda && \text{(application)} \\ & | \lambda x.\Lambda && \text{(abstraction)}\end{aligned}$$

Definition (β -conversion).

$$(\lambda x.M)N \rightarrow_{1\beta} M[x/N].$$

Definition (β -reduction).

$\rightarrow_{\beta}$: Closure reflexive and transitive of $\rightarrow_{1\beta}$.

λ -calculus (cont.)

Example.

$I \equiv \lambda x.x$ (The identity operator)

$K \equiv \lambda xy.x$ (The first coordinate projection operator)

$S \equiv \lambda xyz.xz(yz)$ (A stronger composition operator)

For all $M, N, O \in \Lambda$,

$$IM \rightarrow_{\beta} M$$

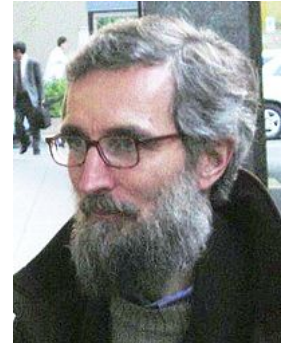
$$KMN \rightarrow_{\beta} M$$

$$SMNO \rightarrow_{\beta} MO(NO)$$

Theorem. In the λ -calculus every (Turing machine)-computable function can be represented by a λ -term (combinator).

Martin-Löf's type theory: Types and terms

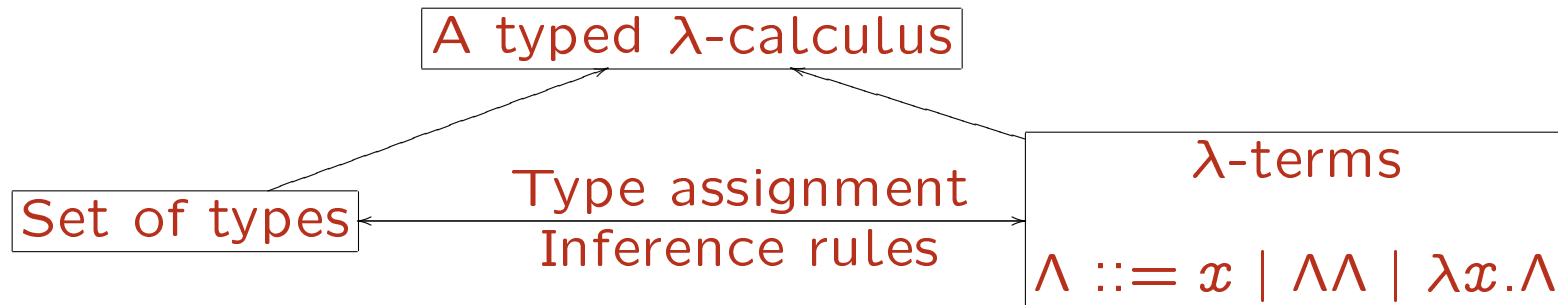
Per Martin-Löf. Swedish logician, philosopher, and mathematician.



Type A	Term $a : A$	
A is a set	a is an element of the set A	$A \neq \emptyset$
A is a proposition	a is a proof (construction) of the proposition A	A is true
A is a problem	a is a method of solving the problem A	A is solvable
A is a specification	a is a program than meets the specification A	A is satisfiable

The Curry-Howard correspondence: The simple type λ -calculus

The general picture



The Curry-Howard correspondence: The simple type λ -calculus (cont.)

Definition (Types).

$$\begin{aligned} T &::= \sigma && \text{(type variables)} \\ &| T \rightarrow T && \text{(function space)} \end{aligned}$$

Definition (Context).

Γ : Finite set of pairs of the form $\{x_1 : \tau_1, \dots, x_n : \tau_n\}$.

$rg(\Gamma) : \{\tau \in T \mid (x : \tau) \in \Gamma, \text{ for some } x\}$.

The Curry-Howard correspondence: The simple type λ -calculus (cont.)

Definition (Judgement $(\Gamma \vdash M : \tau)$).

1. The λ -term M has the type τ in Γ
2. The program M is a proof of the formula τ in Γ

Theorem (Curry-Howard correspondence).

1. If $\Gamma \vdash M : \tau$, then $rg(\Gamma) \vdash \tau$ in $NJ(\rightarrow)$.
2. If $\Delta \vdash \tau$ in $NJ(\rightarrow)$, then $\Gamma \vdash M : \tau$ for some M and some Γ with $rg(\Gamma) = \Delta$.

The Curry-Howard correspondence: The simple type λ -calculus (cont.)

Definition (Rules).

$$\Gamma, \sigma \vdash \sigma \quad (\text{Ax})$$

$$\Gamma, x : \sigma \vdash x : \sigma \quad (\text{Var})$$

$$\frac{\Gamma, \sigma \vdash \tau}{\Gamma \vdash \sigma \rightarrow \tau} \quad (\rightarrow I)$$

$$\frac{\Gamma, x : \sigma \vdash y : \tau}{\Gamma \vdash \lambda x. y : \sigma \rightarrow \tau} \quad (\text{Abs})$$

$$\frac{\Gamma \vdash \sigma \rightarrow \tau \quad \Gamma \vdash \sigma}{\Gamma \vdash \tau} \quad (\rightarrow E)$$

$$\frac{\Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash MN : \tau} \quad (\text{App})$$

The Curry-Howard correspondence: The simple type λ -calculus (cont.)

Example.

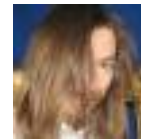
$$\Gamma = \{x : \sigma \rightarrow \tau \rightarrow \rho, y : \sigma \rightarrow \tau, z : \sigma\}$$

$$\begin{array}{c} \frac{\Gamma \vdash x : \sigma \rightarrow \tau \rightarrow \rho \quad \Gamma \vdash z : \sigma}{\Gamma \vdash xz : \tau \rightarrow \rho} \text{(App)} \quad \frac{\Gamma \vdash y : \sigma \rightarrow \tau \quad \Gamma \vdash z : \sigma}{\Gamma \vdash yz : \tau} \text{(App)} \\ \hline \frac{\Gamma \vdash xz : \tau \rightarrow \rho \quad \Gamma \vdash yz : \tau}{\Gamma \vdash xz(yz) : \rho} \text{(App)} \\ \hline \frac{\Gamma \vdash xz(yz) : \rho}{x : \sigma \rightarrow \tau \rightarrow \rho, y : \sigma \rightarrow \tau \vdash \lambda z. xz(yz) : \sigma \rightarrow \rho} \text{(Abs)} \\ \hline \frac{x : \sigma \rightarrow \tau \rightarrow \rho, y : \sigma \rightarrow \tau \vdash \lambda z. xz(yz) : \sigma \rightarrow \rho}{x : \sigma \rightarrow \tau \rightarrow \rho \vdash \lambda yz. xz(yz) : (\sigma \rightarrow \tau) \rightarrow \sigma \rightarrow \rho} \text{(Abs)} \\ \hline \vdash \lambda xyz. xz(yz) : (\sigma \rightarrow \tau \rightarrow \rho) \rightarrow (\sigma \rightarrow \tau) \rightarrow \sigma \rightarrow \rho \text{(Abs)} \end{array}$$

Agda

From: Chalmers University of Technology, Sweden

Agda.cabal.Author: Ulf Norell, Nils Anders Danielsson, Catarina Coquand, Makoto Takeyama, Andreas Abel, ...



Agda as an interactive theorem prover

(From A. Setzer. Interactive Theorem Proving for Agda Users)

Interactive Theorem Proving

- Proofs are fully checked by the system
- Proof steps have to be carried out by the user
- Advantages:
 - Correctness guaranteed (provided the theorem prover is correct)
 - Everything which can be proved by hand, should be possible to be proved in such systems
- Disadvantages:
 - It takes much longer than proving by hand. Similar to programming.
 - Requires experts in theorem proving

Agda as an interactive theorem prover (cont.)

Agda's core: The type *Set* and the dependent function types $(x : A) \rightarrow B$ (Martin-Löf's logical framework.)

Agda code: See file src/eg.agda

Conclusions

logic	typed λ -calculus
formula	type
propositional variable	type variable
implication	function space
proof	λ -term
assumption	object variable
introduction	constructor
elimination	destructor
normal proof	normal form
provability	inhabitation

(M.-H. Sørensen and P. Urzyczyn. *Lectures on the Curry-Howard isomorphism*, volume 149 of *Studies in Logic and the Foundations of Mathematics*. Elsevier, 2006, p. 89)

References

- Agda

- The Agda wiki: <http://wiki.portal.chalmers.se/agda/>
- A. Bove and P. Dybjer. Dependent types at work. In A. Bove et al., editors, *LerNet ALFA Summer School 2008*, volume 5520 of *LNCS*, pages 57–99, 2009.
- U. Norell. Dependently typed programming in Agda. Eprint: wiki.portal.chalmers.se/agda/, 2008.

- Intuitionist logic

D. van Dalen. *Logic and structure*. Springer, 4th edition, 2004.

References (cont.)

- Martin-Löf type theories
 - P. Martin-Löf. *Intuitionistic type theory*. Naples, Bibliopolis, 1984. Notes by Giovanni Sambin of a series of lectures given in Padua, June 1980.
 - B. Nordström, K. Pettersson, and J. M. Smith. *Programming in Martin-Löf's Type Theory*. Oxford University Press, 1990.

References (cont.)

- The Curry-Howard correspondence

M.-H. Sørensen and P. Urzyczyn. *Lectures on the Curry-Howard isomorphism*, volume 149 of *Studies in Logic and the Foundations of Mathematics*. Elsevier, 2006

- The λ -calculus

- J. R. Hindley and J. Seldin. *Lambda-calculus and Combinators. An Introduction*. Cambridge University Press, 2008.

- H. Barendregt. *The Lambda Calculus. Its Syntax and Semantics*. Elsevier, revised edition, 2004. 6th. impression.

- The typed λ -calculus

H. Barendregt. Lambda calculi with types. In S. Abramsky et al., editors, *Handbook of Logic in Computer Science*, volume 2, pages 117–309. Oxford University Press, 1992.